Rank Transform

1 Definition

Rank transform, also called *ranking* or *ordinal transform*, consists in replacing each numerical value by its rank when the data is sorted. The smallest value receives rank 1, the second smallest rank 2, and so on. In this way, the original numerical variable is transformed into an ordinal one.

The key idea is that rank transform preserves the *relative ordering* of observations, but discards information about their exact magnitudes.

2 Why Use Rank Transform?

Rank transform is useful for several reasons:

Resistance to outliers: extreme values only receive extreme ranks and no longer dominate the scale;
Distribution-free transformation: it can be applied regardless of the original distribution;
Comparability: features with very different units or scales become comparable after ranking;
textbfNon-parametric analysis: many statistical methods rely only on ranks rather than on raw values;
Reduction of skewness: rank-based data is much less sensitive to highly skewed distributions.

3 Basic Principle

Given values $x_1, \ldots, x_n$, the rank transform replaces each x_i by its position in the sorted sample. For example, for

$$[50, 30, 90, 10, 70],$$

the sorted values are

$$[10, 30, 50, 70, 90],$$

so the transformed data is

$$[3, 2, 5, 1, 4].$$

Thus, rank transform keeps the order between observations, but not the distances between them.

4 Handling Ties

When several values are equal, one must choose how to assign their ranks. Common methods include:

Average rank: tied values receive the average of the ranks they occupy;
Minimum rank: tied values receive the smallest of these ranks;
Maximum rank: tied values receive the largest of these ranks;
Dense rank: ties share the same rank and the next distinct value gets the next integer;
Ordinal ranking: ties are broken according to their order of appearance.
The average-rank method is the most common in statistics.

5 Normalized Ranks

Instead of raw ranks, one may use normalized ranks:

$$r_i^{\text{norm}} = \frac{r_i - 1}{n - 1},$$

which map the data approximately into the interval $[0, 1]$. This is closely related to the empirical cumulative distribution function and to quantile transformations.

6 Properties

Rank transform has the following important properties:
- it preserves the ordering of the data;
- it is highly robust to outliers;
- it destroys distance information;
- it is not reversible: the original values cannot be recovered from the ranks;
- it preserves monotonic relationships, but may distort non-monotonic ones.

For instance, the datasets

$$[1, 2, 3], \qquad [10, 20, 30], \qquad [1, 1000, 1000000]$$

all produce the same rank representation:

$$[1, 2, 3].$$

7 Comparison with Other Scaling Methods

Unlike standardization or min-max scaling, rank transform does not preserve numerical distances. It only preserves order.

Standardization.

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

uses the mean and standard deviation, and is sensitive to outliers.

Min-max scaling.

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

maps values to a fixed interval, but can be heavily distorted by a single extreme observation.

Rank transform.

$$x_{\text{rank}} = \text{rank}(x)$$

is robust to outliers, but loses all magnitude information.

8 Worked Example

Consider the data

$$[15, 8, 42, 23, 8, 100, 31].$$

First, sort the values:

$$[8, 8, 15, 23, 31, 42, 100].$$

Mapping back to the original order gives

$$[3,\ 1.5,\ 6,\ 4,\ 1.5,\ 7,\ 5].$$

9 Applications

Rank transform appears in many settings: non-parametric statistics, such as Spearman correlation and Mann–Whitney tests; preprocessing for machine learning when outliers are present; comparison of variables measured on different scales; quantile-based transformations; robust feature engineering in heavy-tailed data.

10 Practical Remarks

In practice, rank transform is usually applied *feature-wise*, that is, independently on each column of a dataset. It is important to:

- choose and document a tie-handling method;

- avoid recomputing ranks on test data using test information alone if reproducibility matters;
- remember that ranking may improve robustness, but at the cost of losing magnitude information.

11 Python Code

Below is a simple implementation of rank transform in Python using average ranks for ties:

```python
def rank_transform(values):
    if not values:
        return []

    sorted_vals = sorted((v, i) for i, v in enumerate(values))
    ranks = [0] * len(values)

    i = 0
    while i < len(sorted_vals):
        j = i
        while j < len(sorted_vals) and sorted_vals[j][0] == sorted_vals[i][0]:
            j += 1

        avg_rank = (i + 1 + j) / 2.0

        for k in range(i, j):
            original_index = sorted_vals[k][1]
            ranks[original_index] = avg_rank

        i = j

    return ranks
```

This implementation is simple and appropriate for educational purposes. In practice, libraries such as `scipy.stats` or `pandas` provide optimized ranking functions with several tie-handling options.